Checking Consumer-Relative State Sufficiency

Exact Quotients and History Counterexamples for Event-Anchored Executions

WuJun Chen
Independent Researcher
2026

This paper is Paper XL of the RIME program. It studies a domain-specific procedure for checking consumer-relative state sufficiency in event-anchored executions. It checks supplied consumers and candidate states, not a universal notion of optimal compression.

Abstract

Problem. A compressed state may determine a consumer's current output without determining its legal commands or their successors. We study this distinction for event-anchored consumers of finite coalescing packet executions.
Approach. A consumer declares typed reports and guarded raw actions; a candidate state is a typed expression over the current partition, a selected fusion's origin, and its subsequent absorption sequence. We give exact read, guard, and enabled-successor conditions for deterministic partial commands. A finite generated-log construction checks these conditions at fixed parameters without an execution-word cutoff.
Results. Rejection produces two registered histories witnessing a read, guard, or successor failure. A successful packet-only compression and three failure mechanisms show why sufficiency is consumer-relative. A local JSON-based implementation evaluates the declared syntax, checks the generated domain, and separately replays rejection histories. Previously established forest and placed union-history realizations support accepted consumers.
Boundary. The reported tests do not machine-prove implementation correctness. No minimal-state synthesis, efficiency improvement, or unrestricted history-consumer theorem is claimed.

Keywords: state sufficiency; observed partial systems; state quotient; event-anchored consumer; counterexample generation; generated-domain verification.

Notation Table

Symbol	Meaning
Q, Ω, F	finite positions, source atoms, and selected fresh block
p, d	labelled cycle and rank-$(n-1)$ defect
h, P_h	chronological raw history and its placed packet endpoint
$\Gamma(h, F), S_F$	selected causal state and its registered generated domain
D, q	declared consumer and candidate state expression
Φ_a	actual causal-state successor for raw letter a
R/G/T	read, guard, and enabled-successor fiber conditions

1 Introduction

Suppose an execution has a current state containing both present packets and selected historical information. A proposed compression keeps only a value $q(x)$. The relevant question is not whether this value summarizes the execution, but whether it suffices for a particular consumer:

> From $q(x)$ alone, can the consumer determine every declared report, decide which requested commands are enabled, and compute the next compressed state after an enabled command?

These are different obligations. A report can be well-defined while a guard is not. Reports and guards can both descend while the next compressed state depends on information already discarded.

The distinction is visible in a three-position example. Let p rotate $\mathbb{Z}_3$, let $d = (0, 2, 0)$, and start with identity-placed singleton atoms. The histories dd and pdd end with the same packet containing all atoms at position zero. Select the final fusion by its fresh block $F = \{0, 1, 2\}$. The compression $q = (P, F)$ supports a consumer reporting current packets and using the usual carrier-based commands. It does not support a consumer, on the same generated domain and with the same command menu, that reports the selected origin's parent at port zero: the answers are $\{0, 2\}$ and $\{1, 2\}$. Section 5 gives the calculation.

Paper XXXIX [2, Theorem 4.1 and Proposition 5.1] establishes the event-anchored causal state and its forest and placed union-history realizations. This paper takes those objects as inputs and checks whether a further candidate compression q supports a declared consumer. The origin-parent witness is reused in Section 5.2 to test that new question.

1.1 Contributions

We provide a domain-specific, witness-producing procedure for checking consumer-relative state sufficiency in event-anchored executions. Its contribution is the exact generated-history construction and its integration with typed candidate expressions and replayable failure explanations, rather than a new general quotient criterion.

The resulting interface is

$$\begin{aligned}(\text{environment}, F, D, q) &\longmapsto S_F \text{ with generating histories} \\ &\longmapsto \text{sufficiency decision or history witness.}\end{aligned}$$

Three ingredients are retained together:

- **Generated-domain coverage.** The states come from actual singleton-seed executions with the selected event registered, not arbitrary well-formed tuples or samples below a word-length cutoff. The finite fusion structure is inherited; the log-only construction supplies the generated image and witnessing histories.
- **Candidates as inputs.** Reports, guards and q are supplied in the declared typed syntax. They may change without changing the checker, so the interface is not a collection of hand-written candidate tests.
- **Source-semantic explanations.** Rejection returns two actual histories in the same environment with the same F and identifies whether the lost information affects a report, enabledness or a successor. It does not stop at an unexplained pair of internal state identifiers.

A noninjective packet-state compression and three failure mechanisms make the criterion concrete. The checker tests a supplied candidate rather than synthesizing a smallest state. The bounded implementation evidence is stated in Section 6.

Section 2 fixes the generated domain and typed expressions. Section 3 proves the exact criterion, Section 4 gives the witness-producing finite procedure, and Section 5 separates successful compression from three failure mechanisms. Section 6 states the evidence scope; Appendices A and B give supplementary constructions.

Related Work and Novelty Boundary

Exact quotients. Rutten treats system morphisms and quotients through commuting transition structures [7, Sections 2 and 4.2]. Theorem 3.1 is an instance for an observed deterministic

partial system: retain the report tuple and, for each label, either an undefined tag or its successor. R, G, and T are exactly the components that must be constant after mapping to a q-fiber. The theorem does not introduce a new general quotient mechanism.

Simulation and continuous execution. Lynch and Vaandrager develop refinements and forward simulations, including extension of step correspondences along executions [6, Sections 3.1-3.2]. Corollary 3.2 uses that standard reasoning with one raw step per command. Here enabledness is also reflected, as proved by G; this is not a property silently inherited from every forward simulation. Continuing with each actual successor is a concrete application, not a new universal path-lifting theory.

Relative exactness and finite checking. Giacobazzi, Ranzato, and Scozzari study completeness relative to an abstraction and concrete operations [4, Introduction]. That is relevant to the demand for an exact successor, rather than merely a sound approximation; no quantale or abstract-domain completion theorem is imported here. Once a finite generated system is available, exploration and fiber comparison are elementary finite-state checks. The model-specific obligation in Lemma 2.2 and Theorem 4.1 is to supply that generated domain despite unbounded execution-word length, preserve actual registration, and recover histories for rejected pairs. We claim neither a new general verification algorithm nor a minimization or complexity improvement.

Counterexample diagnostics. CADP's BISIMULATOR already compares labelled transition systems modulo equivalences or preorders and can return an explanatory diagnostic when a comparison fails [1, Description and option -diag]. Returning a counterexample is therefore not itself a new contribution. The specific connection here is between event registration, typed candidate expressions and replayable source histories for a failed R/G/T clause. No reduction to BISIMULATOR, comparative benchmark or general diagnostic superiority is asserted.

The following compact comparison concerns documented interfaces, not measured performance or an expressivity separation. mCRL2's `ltscompare` also supports equivalence/preorder checks and counterexamples for specified equivalences [9, Usage and option –counter-example].

Tool	Declared input	Result relevant here
CADP	Two represented labelled transition systems	Equivalence/preorder verdict; optional BISIMULATOR diagnostic on failure
mCRL2	Two nonprobabilistic LTS inputs	Equivalence/preorder verdict; `ltscompare` counterexamples for supported equivalences
Q2 in this paper	Environment, registered-event argument, typed consumer and candidate q	Generated-domain R/G/T check; two actual registered histories on failure

The domain-specific work in Q2 is constructing that history domain and connecting the supplied expressions to source-level explanations. The other tools are not run in this study. No translation to them, inability to encode the same task, or advantage in speed, scale or diagnostic quality is claimed.

Folds. Hutton develops standard fold proof principles, fusion, and paired computations [5, Sections 2-4]. Structural interpretation and the use of folds are not novelty claims here. Our chronological left-fold and snoc equation are proved directly in Appendix A; Hutton's right-fold equations are not quoted as an identical update rule. The streaming condition is sufficient, not a characterization of all incremental queries.

Union histories and UFE. Stevens and Ghidini retain effective-union history and verify explanation algorithms, their extensional agreement, and an imperative refinement with total correctness [8, Sections 3-5]. Their state is not merely a current partition or a single explanation output. XXXIX's placed forward adapter is positive compatibility with that line of representation, not an expressivity separation. This manuscript composes with that adapter;

it does not inherit Isabelle verification or performance guarantees for its additional placement and consumer machinery.

2 Generated Causal States and Typed Consumers

2.1 Fixed environment and registered events

Fix $Q = \mathbb{Z}/n\mathbb{Z}$, a finite atom set Ω with $2 \le N = |\Omega| \le n$, and an injective singleton seed $\iota : \Omega \hookrightarrow Q$. The labelled actions are the cycle $p(q) = q + 1$ and one rank-$(n-1)$ map d. Its unique binary kernel has fixed ordered ports (k_0, k_1) and collision image c.

A placed partition P assigns disjoint nonempty atom blocks to occupied positions, with union Ω; unoccupied positions have the empty block. A raw letter $a \in \{p, d\}$ updates every packet by

$$(f_{a\#}P)(q') = \bigcup_{q : f_a(q) = q'} P(q).$$

Words are chronological: ha executes h and then a. A fusion joins the packets at the two kernel ports. Its fresh block is their union, and its origin snapshot records

$$o = (d, (k_0, U_0), (k_1, V_0), c).$$

Fix a nonempty invocation argument $F \subseteq \Omega$. A history is registered for F exactly when one of its actual fusions created F. That occurrence is unique: atoms never split, and every later fusion of their carrier adds a disjoint nonempty partner. Registration is not inferred merely from containment in a current packet.

For a registered history h, let

$$\Gamma(h, F) = (P_h, F, o_F, \mathcal{A}_F(h)), \qquad S_F = \{\Gamma(h, F) : h \text{ is registered for } F\}.$$

The list $\mathcal{A}_F(h)$ contains exactly the later fusions involving the selected carrier, in chronological order. Each entry is

$$\rho = (d, q, C, s, B, c, C \cup B),$$

where $C \supseteq F$ was the carrier at port q, and B its partner at the other port s. There are no absolute timestamps or records of plateau-word history. The domain S_F is the generated image, not the set of all well-formed tuples with these fields. It can be empty.

2.2 Closure and finite enumeration

For $x = (P, F, o, A) \in S_F$, let (q, C) be the unique current packet containing F. Define

$$G_a(P, q) = \{s \in \operatorname{supp} P : f_a(s) = f_a(q)\}.$$

The successor $\Phi_a(x)$ replaces P by the full $f_{a\#}P$, keeps F, o, and appends the actual record ρ precisely when $G_a(P, q)$ has two elements. Otherwise it keeps A. An unrelated fusion still updates the full partition.

Lemma 2.1 (inherited causal closure)**.**
Registration persists and

$$\Gamma(ha, F) = \Phi_a(\Gamma(h, F)).$$

Proof. A raw step updates all packets. If it does not merge the selected carrier, it adds no selected absorption. If it does, exactly one disjoint partner joins the carrier and the displayed record is appended. Earlier events are unchanged and atoms do not split. These are exactly the stated successor rules. Induction extends the equality to every future word. This is the causal-state closure already proved in [2], not a new claim of priority. □

Lemma 2.2 (finite generated domain)**.**
The set S_F can be enumerated effectively without bounding raw word length and without using a forest representation.

Proof. Use an auxiliary log state consisting of the full current partition and the chronological list of all fusion events. Each fusion decreases rank, so this list has length at most $N-1$. At fixed Q, Ω, d, every field has a finite alphabet. Nonfusion steps change placement but do not extend the list. Consequently these log states form a finite deterministic system. Breadth-first exploration from the singleton seed to saturation yields exactly the generated log states. Decode Γ from those containing an event with fresh block F. Store predecessor edges outside the semantic state key to recover witnessing histories. The full fusion list is an enumeration device, not a new retained component of Γ. □

2.3 Independent typed expressions

The expression language uses the base sorts

$$\mathsf{Bool}, \mathsf{Nat}, \mathsf{Atom}, \mathsf{Pos}, \mathsf{Letter}, \mathsf{Block}, \mathsf{Partition}, \mathsf{Origin}, \mathsf{AbsRecord},$$

together with products, lists, and options. Its four history-query leaves are

`CurrentPartition` `SelectedBlock` `OriginSnapshot` `AbsorptionSequence`

Finite terms are formed from typed literals and variables, pure total operations, tuples and projections, conditionals, and chronological left folds. Products, records, lists, and options follow the usual typed rules. Write $\mathsf{Entry} = \mathsf{Pos} \times \mathsf{Block}$. The remaining primitive library is fixed as follows; τ denotes any one value sort.

Primitive	Input and output sorts
Equality	$\tau \times \tau \to \mathsf{Bool}$
Negation; conjunction and disjunction	$\mathsf{Bool} \to \mathsf{Bool}$; $\mathsf{Bool}^2 \to \mathsf{Bool}$
Addition; strict and non-strict comparison	$\mathsf{Nat}^2 \to \mathsf{Nat}$; $\mathsf{Nat}^2 \to \mathsf{Bool}$
Remainder modulo a fixed positive integer	$\mathsf{Nat} \to \mathsf{Nat}$
Block union and intersection; inclusion	$\mathsf{Block}^2 \to \mathsf{Block}$; $\mathsf{Block}^2 \to \mathsf{Bool}$
Block cardinality	$\mathsf{Block} \to \mathsf{Nat}$
List prepend	$\tau \times \mathsf{List}(\tau) \to \mathsf{List}(\tau)$
Packet lookup; packet enumeration	$\mathsf{Partition} \times \mathsf{Pos} \to \mathsf{Option}(\mathsf{Block})$; $\mathsf{Partition} \to \mathsf{List}(\mathsf{Entry})$
Carrier lookup	$\mathsf{Partition} \times \mathsf{Block} \to \mathsf{Option}(\mathsf{Entry})$
f_p, f_d	$\mathsf{Pos} \to \mathsf{Pos}$

Packet entries are enumerated in increasing position order. Packet lookup returns None at an empty position; carrier lookup returns None unless there is exactly one containing entry. These are ordinary option values, not undefined commands. Records have precisely the origin and absorption fields of Section 2.1. Origin parent entries are ordered by the fixed kernel ports. In an AbsRecord, the first parent is the selected carrier and the second is its partner; each retains its actual event-time port. Atoms, positions, natural numbers and Booleans remain

different sorts; there are no implicit coercions. The fixed environment and finite literal values may be constants.

A fold over $\mathsf{List}(\tau)$ with accumulator sort σ has a body of sort σ under lexical bindings $\mathsf{acc} : \sigma$, $\mathsf{item} : \tau$. It evaluates the list and seed once and processes the list from left to right. Nested folds are allowed; general recursion, unbounded loops, arbitrary code callbacks, and raw-word or timestamp oracles are not. Reports, guards, and candidate states are closed terms. All primitives are effective and total on their declared sorts. This independently interpreted syntax supplies the factorization problem; forest realizability is not a condition in its definition.

Interpret the four leaves first on execution logs, then separately on the components of Γ.

Lemma 2.3 (structural query factorization)**.**
For every well-typed expression e, registered history, and typed valuation υ,

$$\mathrm{Eval}_{\log}(e, h, F, \upsilon) = \mathrm{Eval}_{\Gamma}(e, \Gamma(h, F), \upsilon).$$

Proof. Induct on typing. Query leaves agree by their log definitions; literals and variables agree immediately. Pure operations preserve equal arguments, and equal conditions select the same branch. For a fold, both the list and initial accumulator agree. Induction along that finite list, applying the body hypothesis at equal accumulator/item valuations, gives equal results. □

A consumer D declares finitely many closed report expressions r_b and finitely many command labels α, each assigned a raw letter a_α and a closed Boolean guard g_α. Distinct labels remain distinct even when their effects agree. On registered histories, an enabled command executes that letter on the same history and keeps the same F. Thus on S_F,

$$\Delta_\alpha(x) = \Phi_{a_\alpha}(x) \quad \text{exactly when } g_\alpha(x) = \mathsf{true}.$$

The default state is Γ, not just the displayed report. This class has no additional private memory or changing infinite external input domain.

3 An Exact Criterion for a Candidate State

Let $q : S_F \to V$ be the evaluation of a closed typed state expression and put $S_q = q(S_F)$. All conditions below quantify over that generated image and the fixed consumer D:

- **R — reports:** if $q(x) = q(y)$, every declared report has equal values at x, y.
- **G — guards:** if $q(x) = q(y)$, each command has equal enabledness at x, y.
- **T — enabled successors:** if $q(x) = q(y)$ and command α is enabled at both, then

$$q(\Phi_{a_\alpha}(x)) = q(\Phi_{a_\alpha}(y)).$$

Theorem 3.1 (consumer-relative state sufficiency)**.**
R, G, and T hold if and only if there are report functions and deterministic partial commands on S_q for which q preserves reports, reflects command domains in both directions, and commutes with enabled successors.

Proof. Suppose R/G/T hold. At $v \in S_q$, choose x with $q(x) = v$. Define the coarse reports to be $r_b(x)$, the domain of a coarse command by $g_\alpha(x)$, and its enabled successor to be $q(\Phi_{a_\alpha}(x))$. R, G, and T respectively make these definitions independent of the representative. Lemma 2.1 puts the successor in S_q. The stated equalities and both directions of definedness follow.

Conversely, equal q-values have equal coarse reports and equal membership in every coarse command domain. When enabled, the same deterministic coarse successor must equal both fine successor images. These are R, G, and T. □

This is the exact observed partial-system quotient mechanism discussed in Related Work and Novelty Boundary, not a new general quotient theorem [7]. The proof above specifies its read, definedness, and successor obligations for this consumer domain.

T is required only on enabled commands. Requiring congruence under every raw letter, including those the consumer disables, is a stronger sufficient condition, not the necessary criterion.

Corollary 3.2 (continuous execution)**.**

Every legal coarse command sequence lifts from every actual registered representative of its initial state, using that representative's actual successive raw updates. Intermediate reports and compressed states agree, with the same F; legality is reflected in both directions.

Proof. Apply domain reflection and successor commutation at each step. The actual successor is the next representative, and registration persists. No representative is reselected between steps. □

The induction is the standard extension of step correspondence to executions [6]; the necessary domain reflection has been established explicitly in Theorem 3.1.

Quantifier boundary. This theorem fixes the consumer before testing q. If one q had to preserve every closed report in the language, the base queries alone would give

$$q(x) = q(y) \Longrightarrow (P_x, F, o_x, A_x) = (P_y, F, o_y, A_y) \Longrightarrow x = y.$$

Such a q would be injective on S_F. This is a consequence of the chosen read signature, not a global minimality theorem for forests or other representations. Nontrivial compression becomes possible because a particular consumer asks for less.

3.1 Inherited representation consequence

Let J_Γ be the forest decoder of Paper XXXIX [2], restricted to the registered generated domain. It is surjective onto S_F and satisfies

$$J_\Gamma \delta_a = \Phi_a J_\Gamma.$$

For an accepted candidate, $J_D = q \circ J_\Gamma$ computes the coarse state. Theorem 3.1 makes its reports, command domains, and enabled successors well-defined; evaluating a guard and applying the existing forest action gives domain reflection and successor commutation. Corollary 3.2 therefore holds on the same actual invocation. The published placed forward-UFE decoder gives the corresponding adapter realization. Neither representation is needed to enumerate S_F for Theorem 4.1.

4 Finite Checking and History Counterexamples

Theorem 4.1 (effective checking)**.**

For a fixed finite environment, fixed F, finite well-typed consumer D, and typed q, satisfaction of R/G/T is decidable. A negative decision has two actual registered histories witnessing the failed clause.

Proof. Lemma 2.2 enumerates the finite generated domain and supplies a generating history for every member. Evaluate the terminating expressions for q, reports, and guards, and group states by their typed q-values. Check R and G within each fiber, then T for commonly enabled commands. Every value computed is finite, even if its declared sort is Nat or a list. These finite checks terminate, and Theorem 3.1 proves their correctness. A failed pair retains its two generating histories; for T append the same enabled raw letter to obtain the two actual successor histories. □

The decision procedure has the following structure:

```
check(environment, F, consumer D, candidate q):
    statically type-check the entire request
    saturate the independent raw log system from the singleton seed
    decode the exact registered causal image S_F
    if S_F is empty: return EMPTY_REGISTERED_DOMAIN
    group S_F by typed q-value, retaining generating histories
    check reports within each fiber; return FAIL_READ on a conflict
    check guards within each fiber; return FAIL_GUARD on a conflict
    check each commonly enabled successor; return FAIL_SUCCESSOR on a conflict
    return PASS_EXACT
```

A rejection identifies the same environment and F, two histories h, h', their equal q-values, and a report or command exposing the discrepancy. For T it also identifies the common enabled command and the two successor q-values. A witness need not be shortest or equal to a preselected example.

Acceptance and rejection have different verification requirements. An exact acceptance requires completeness of the generated domain and correctness of all relevant checks. A valid two-history rejection establishes its separation without certifying domain completeness. A separate verifier can replay the histories without trusting producer state IDs.

The local implementation imposes resource limits. It reports `INVALID_INPUT` for malformed requests and `UNRESOLVED_LIMIT` when saturation or evaluation is interrupted. Neither is a completed mathematical decision. Empty registration is a vacuous instance, not event supply. The mathematical procedure has no word-length cutoff, but it carries no efficiency guarantee. Its executable scope and supporting records are described in Section 6.

4.1 An illustrative request

To isolate the read failure of Section 5.2, supply

$$n = 3, \quad d = (0, 2, 0), \quad \iota = \mathrm{id}, \quad F = \Omega,$$
$$q(x) = (P_x, F), \quad r(x) = \text{origin parent at port } 0, \quad \mathsf{Cmd} = \emptyset.$$

In the retained run, the checker returns a read failure with histories dd and pdd, equal q-values, and report blocks $\{0, 2\}$ and $\{1, 2\}$. Section 5.2 proves this separation. The experiment README supplies the complete typed JSON and replay commands; no executable configuration is needed to read the proof.

5 One Successful Compression and Three Failure Mechanisms

All examples below use actual histories from their declared singleton seed. Their task specifications are essential: different consumers can give different answers for the same candidate.

5.1 Successful packet-only compression

Use $q_P(P, F, o, A) = (P, F)$. Let the reports be current packet data and let the commands be the Track menu: `Carry(a)` when the selected carrier's collision group has size one, and `Absorb(a)` when it has size two.

Proposition 5.1.
The candidate q_P is sufficient for this consumer on every declared registered domain.

Proof. Reports are functions of P, F. These values also determine the unique carrier and its collision group, so every guard descends. Every raw successor has compressed value $(f_{a\#}P, F)$, independent of the discarded origin and absorption list. Thus R/G/T hold. □

This is genuinely noninjective in the three-position example from the introduction: the two histories below have different origins but the same q_P. The result is not merely a check of the identity encoding $q = \Gamma$.

5.2 Read failure: same candidate, different requested report

Let $Q = \Omega = \mathbb{Z}_3$, $d = (0, 2, 0)$, and $F = \Omega$. Compare $h = dd$ with $h' = pdd$. Immediately before the final d:

History	Parent at port 0	Parent at port 2
dd	$\{0, 2\}$	$\{1\}$
pdd	$\{1, 2\}$	$\{0\}$

Both final partitions are $0 : \{0, 1, 2\}$. Both selected absorption lists are empty, and both carriers are the same. Keep q_P and the same Track command menu, but request the origin parent at port zero. Then

$$q_P(\Gamma(h, F)) = q_P(\Gamma(h', F)), \qquad r(\Gamma(h, F)) \neq r(\Gamma(h', F)).$$

R fails. The successful and rejected checks differ only in what is reported, not in the environment or candidate. This origin counterexample is inherited from [2].

5.3 Guard failure: carrier-local data omit a relevant packet

Let $Q = \Omega = \mathbb{Z}_4$, $d = (0, 0, 1, 2)$, and $F = \{0, 1\}$. Compare $h = dp$ and $h' = dppd$. Their partitions are

$$P_h = \{1 : F,\ 2 : \{2\},\ 3 : \{3\}\},$$
$$P_{h'} = \{0 : \{3\},\ 1 : F,\ 2 : \{2\}\}.$$

In both histories, the first d created F from $\{0\}$ at port zero and $\{1\}$ at port one. Their selected absorption lists are empty and their current carriers are $(1, F)$. Hence

$$q_{\mathrm{loc}}(x) = (o_F, \mathcal{A}_F, \mathrm{carrier}(x))$$

has the same value on both histories. Take these fields as the report and use the Track commands. In P_h, port zero is empty, so `Absorb(d)` is disabled. In $P_{h'}$, port zero contains $\{3\}$, so the same command is enabled. G fails. The omitted packet is irrelevant to the present carrier report but relevant to its next command domain.

5.4 Successor failure: a readable parity is not a sufficient state

Use the same four-position histories, but a different consumer. Let

$$q_{\mathrm{par}}(x) = |\mathcal{A}_F(x)| \bmod 2,$$

report that parity, and provide one always-enabled command with raw effect d. Both initial parities are zero, so reports and guards agree. After the command, dpd has no selected absorption, whereas $dppdd$ has absorbed $\{3\}$ into F. Consequently

$$q_{\mathrm{par}}(\Gamma(dpd, F)) = 0, \qquad q_{\mathrm{par}}(\Gamma(dppdd, F)) = 1.$$

T fails. The guard and parity examples reuse one generated-history pair with different consumer specifications. They are not a claim that one candidate violates exactly one condition, nor a separate general independence theorem for R/G/T.

The supplementary two-position control compares d and pd for $d = (0, 0)$. It retains the same unordered origin parents but exchanges their port assignments. Its lost information is the **parent-to-port binding**, not fixed port numbers that can already be recovered from the environment. This auxiliary control is retained in the Section 6 bounded record; the three main failures are proved above.

6 Computational Artifacts and Evidence Scope

The paper-owned package is under `experiments/paper40/` in the RIME repository. Its README supplies the complete typed request, language specification and replay commands. The source mirror binds the maintained `sof-runtime` implementation and the exact Paper XXXIX inputs used by the differential tests [2]. The extracted snapshot replays with Python 3.11+ and its standard library, without Git or either development checkout.

The local receipt binds package bytes and extracted-source replay. It does not verify a public tag, downloaded bytes, or an independent implementation; those require separate external checks.

The rejection replay reconstructs both histories without producer transition, decoder or saturation functions, but shares record types and expression interpretation with the checker. It certifies a displayed separation, not complete domain exploration.

The retained record reports 30 passing application tests with no skips: 24 syntax-wide tests and six earlier fixed-control tests. The nine specified program/candidate combinations give five exact positives, two read failures, one guard failure, and one successor failure. All four rejections pass the separate replay. The exhausted environments are:

Positions	Defect	Selected F	Generated log states	Registered causal states
2	$(0, 0)$	$\{0, 1\}$	6	4
3	$(0, 2, 0)$	$\{0, 1, 2\}$	39	18
4	$(0, 0, 1, 2)$	$\{0, 1\}$	292	72

Tests cover typing, folds, forged payloads, resource limits and the frozen-source comparison. One seven-position continuation is checked, not an exhausted seven-position domain. Separate packaging checks concern integrity, not additional mathematical cases. The tests are bounded consistency controls; Sections 2-5 contain the mathematical proofs.

Short path	Role and evidence boundary
`README.md`	Reader commands and complete typed request
`source/q2-source-v1.zip`	Runnable source mirror, not another implementation owner
`source/snapshot.v1.json`	Exact archive and source-commit binding
`results/q2-controls.v1.json`	Requests, decisions, rejection replay and bounded test record
`release-manifest.json`	Exact manuscript, PDF and evidence inventory
`validation/validate_release.py`	Read-only release-content closure check
`results/`	Local receipt; exact path in manifest, not self-bound

The manifest enumerates the remaining bound files, including the environment record and package validator. The same checker accepts different declared consumers and candidates within this environment family; the bounded runs do not establish cross-domain utility or improved performance.

Claim Status and Boundary

Item	Status and scope
Generated causal closure	Lemma 2.1; inherited from Paper XXXIX
Finite generated-domain enumeration	Lemma 2.2; paper proof, no raw-word cutoff
Read/guard/successor quotient criterion and continuation	Theorem 3.1 and Corollary 3.2; exact on the declared generated domain
Typed finite decision and rejection histories	Theorem 4.1; mathematical construction for fixed finite parameters
Packet-only positive and three negative controls	Section 5; exact examples for specified consumers and candidates
Source snapshot and bounded implementation replay	Local consistency and closure checks; no machine proof of the general results
Minimal state, efficiency, or unrestricted consumer adequacy	Not established

7 Conclusion

State sufficiency is relative to a consumer's reports, guards, and enabled successor requirements. For the declared event-anchored execution domain, these obligations have an exact quotient criterion and an effective finite check with actual-history rejection witnesses. The same packet compression succeeds for one consumer and fails for another; omitted global packets and non-incremental summaries exhibit distinct guard and successor failures.

The result checks a supplied candidate on a fixed generated domain; the inherited forest and placed union-history representations provide compatible realizations without entering the decision algorithm. Further minimization or unrestricted consumer classes require separate results. The organizing question is whether the declared consumer can continue exactly from the proposed state, and which actual histories refute sufficiency if it cannot.

Appendix A A Sufficient Streaming Schema and Closure Rules

The typed language allows a fold body to read the current partition. A previously computed fold value need not determine its recomputation after that partition changes, even if no selected absorption is appended. Structural query factorization alone is not a streaming-state theorem.

A sufficient restricted construction is available. Let $b_0(F, o)$ and $B(F, o; v, \rho)$ be typed value expressions whose only varying inputs are the immutable F, o, accumulator v, and current record ρ. They have no ambient history-query leaves. Set

$$v(x) = \mathrm{fold}(A, b_0(F, o); B), \qquad q_B(x) = (P, F, o, v(x)).$$

Proposition A.1 (immutable-context fold)**.** *This candidate has a deterministic successor under every raw letter and supports every consumer whose reports and guards are typed functions of its displayed components.*

Proof. The fold/snoc identity is

$$\mathrm{fold}(A \parallel [\rho], b_0; B) = B(F, o; \mathrm{fold}(A, b_0; B), \rho).$$

The current P, F determine whether an absorption occurs and every field of its record. Compute the full P'; keep v if there is no selected absorption, otherwise apply B. The immutable context is unchanged. This proves successor congruence, while the stated report/guard restriction gives R/G. Apply Theorem 3.1. □

Finite products and typed postcomposition of reports preserve sufficiency. Restricting a guard by a Boolean function of the descended state preserves it too. For two admitted consumers executing the same raw letter jointly, the product domain is the correlated image

$$\{(q_1(x), q_2(x)) : x \in S_F\},$$

not the unrestricted Cartesian product. With both component guards true, each next component is computed from the same $\Phi_a(x)$, proving the product's R/G/T conditions. Sequential execution uses actual successors; it is not rollback or an atomic transaction. Postcomposing state encodings, unlike report values, can destroy successor congruence, as the projection to parity in Section 5.4 illustrates.

Appendix B Forward Operators and Fiber Kernels

This appendix gives a supplementary forward reformulation with its proof; it is not used by Q2.

Let X_F be the finite nonempty set of registered seed-generated forest invocations and let $J_\Gamma : X_F \twoheadrightarrow S_F$ be the inherited decoder. For a command α, define its partial-action matrix on the orthonormal state basis by

$$T_\alpha e_x = \begin{cases} e_{\delta_{a_\alpha} x}, & g_\alpha(J_\Gamma x) = 1, \\ 0, & g_\alpha(J_\Gamma x) = 0. \end{cases}$$

Equip $\mathbb{C}[X_F]$ with that orthonormal basis. If a partition $X_F = \bigsqcup_i S_i$ into nonempty sectors is declared, the coordinate projectors $Q_i e_x = \mathbf{1}_{x \in S_i} e_x$, together with the labelled family T_α, give an operator SOF under the abstract definition of [3]. The sectorization is additional declared data, not a canonical choice determined by the forest. The transitions need not be unitary, and equal matrices do not identify distinct command labels. This construction uses neither a finite-permutation conformance claim nor a reporting adapter.

Zero represents an undefined command, not an added physical state. Set

$$L_q e_x = e_{q(J_\Gamma x)}, \qquad E_{b,t} e_x = \mathbf{1}_{r_b(J_\Gamma x) = t} e_x.$$

Proposition B.1 (equivalent forward formulations)**.** *R/G/T are equivalent to the existence of descended report-value projectors and deterministic partial-action matrices satisfying*

$$L_q E_{b,t} = \overline{E}_{b,t} L_q, \qquad L_q T_\alpha = \overline{T}_\alpha L_q.$$

They are also equivalent to invariance of $K_q = \ker L_q$ under all the displayed fine operators, where

$$K_q = \operatorname{span}\{e_x - e_y : q(J_\Gamma x) = q(J_\Gamma y)\}, \qquad \dim K_q = |X_F| - |S_q|.$$

Proof. Under R/G/T define the coarse data as in Theorem 3.1 and verify the identities on each basis vector. Conversely, surjectivity of J_Γ lifts every pair in a q-fiber to the forest domain. A report disagreement gives a nonzero basis vector versus zero under a suitable report projector. Unequal enabledness does the same under a command; distinct enabled coarse successors give different basis vectors. The intertwining identities exclude all three failures.

The kernel consists of vectors whose coefficients sum to zero in each fiber. Differences from a selected pivot form a basis of each fiber's zero-sum subspace. Kernel invariance is equivalent to a well-defined linear map on the quotient. For these particular operators, it also enforces the required report-projector and zero-or-single-basis-column forms, by the same difference-vector test. □

Report projectors are self-adjoint, so their kernel invariance makes the kernel reducing for those projectors. Transition-adjoint invariance does not follow. For example,

$$L = (1\ 1), \quad T = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \quad \overline{T} = (1)$$

satisfy $LT = \overline{T}L$, but $LT^* = (2, 0) \neq (1, 1) = \overline{T}^* L$. No unitary equivalence, strict SOF morphism, spectral equality, or executable inverse is implied. The declared observation SOF remains well-defined even when a candidate fails the autonomous-state criterion: its construction does not require a quotient by that candidate.

References

[1] CADP Project. *BISIMULATOR Manual Page*. INRIA. Online manual; Description and option -diag; no publication year inferred from access date.

[2] WuJun Chen. Event-anchored fusion ancestry and consumer-adequate continuations: Finite realizations for single-defect circular actions, 2026. RIME Paper XXXIX, version 1.0, Zenodo. DOI: 10.5281/zenodo.23410022.

[3] WuJun Chen. Sectorized observable framework: A typed static object language with exact marked finite realizations, 2026. RIME Paper VIII, version 2.1, Zenodo. DOI: 10.5281/zenodo.21977464; abstract static definition only.

[4] Roberto Giacobazzi, Francesco Ranzato, and Francesca Scozzari. Building complete abstract interpretations in a linear logic-based setting. In *Static Analysis, SAS 1998*, volume 1503 of *Lecture Notes in Computer Science*, pages 215–229. Springer, 1998.

[5] Graham Hutton. A tutorial on the universality and expressiveness of fold. *Journal of Functional Programming*, 9(4):355–372, 1999.

[6] Nancy Lynch and Frits Vaandrager. Forward and backward simulations I. untimed systems. *Information and Computation*, 121(2):214–233, 1995.

[7] J. J. M. M. Rutten. Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80, 2000.

[8] Lukas Stevens and Rebecca Ghidini. Simplified and verified: A second look at a proof-producing union-find algorithm. In *Automated Deduction, CADE 2025*. Springer, 2025. Versioned arXiv full text 2504.10246v2.

[9] Muck van Weerdenburg. *ltscompare: mCRL2 User Manual*. mCRL2 Project. Online manual; Usage and option –counter-example; interface comparison only.